

Elemen Dasar Web: HTML, CSS, Javascript

Mata Kuliah II3140 - Pengembangan Aplikasi Web dan Mobile



Dr. Fadhil Hidayat S.Kom., M.T.



Ir. Devi Willieam Anggara S.T., M.Phil., Ph.D., IPP.



Bahan Kajian

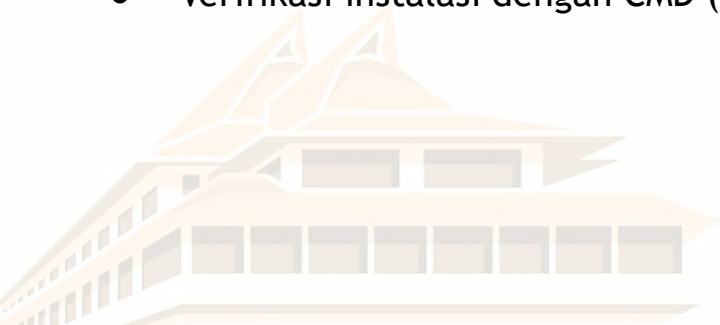


1. Python
2. Setup Flask
3. HTML
4. CSS
5. Javascript



Jika Python belum terinstal di komputer Anda, Anda harus menginstalnya terlebih dahulu agar bisa menjalankan Flask:

- Pada situs resmi python.org unduh versi terbaru untuk OS Anda.
- Buka file installer yang sudah diunduh.
- Add python.exe to PATH, jika ini terlewat, perintah python tidak akan dikenali di Command Prompt. Bisa di-setting di environment variable (Windows).
- Klik Install Now (atau Customize installation jika Anda ingin memindahkan seluruh program utama Python ke drive lain, namun membiarkannya di drive C secara default tetap aman).
- Tunggu hingga proses selesai.
- Verifikasi instalasi dengan CMD (Windows) ketikkan: **python --version**



Mulai Instal Flask di Drive Tertentu



Buka cmd pada windows:

1. Pindah ke drive tujuan (misal Drive D)

D:

2. Buat dan masuk ke folder proyek

mkdir ProyekFlask

cd ProyekFlask

3. Buat virtual environment agar Flask terisolasi di drive ini

python -m venv env_flask

4. Aktifkan environment-nya

env_flask\Scripts\activate

5. Instal dan virifikasi Flask setelah instalasi selesai

pip install Flask

flask --version

- Flask adalah framework pengembangan web mikro (*micro web framework*) yang ditulis menggunakan bahasa pemrograman Python.
- Disebut "mikro" karena Flask sangat ringan, minimalis, dan fleksibel.
- Berbeda dengan Django yang sudah menyediakan semua fitur bawaan (seperti sistem database dan login), Flask hanya menyediakan modul inti saja.
- Anda dibebaskan untuk memilih sendiri tools, database, dan struktur proyek yang ingin Anda gunakan.



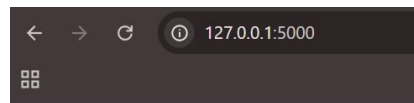
Website Pertama dengan Flask



Buat File Python dan beri nama `app.py`:

```
1. from flask import Flask
2. app = Flask(__name__)
3. @app.route('/')
4. def home():
5.     return """
6.     <html>
7.     <head>
8.     <title>Website Pertama</title>
9.     </head>
10.    <body>
11.    <h1>Selamat Datang</h1>
12.    <p>Halaman web ini dibuat dengan Python Flask.</p>
13.    </body>
14.    </html>
15.    """
16. if __name__ == '__main__':
17.     app.run(debug=True)
```

- Jalankan Aplikasi pada CMD
`python app.py`
- Hasil:
* Running on
`http://127.0.0.1:5000`
- Buka browser dan akses:
`http://127.0.0.1:5000`



Selamat Datang

Halaman web ini dibuat dengan Python Flask.



Pisahkan Kode HTML dari Python

Atur struktur folder menjadi

ProyekFlask/

|

|— app.py

|— templates/

|— index.html

```
1. from flask import Flask,
   render_template
2. app = Flask(__name__)
3. @app.route('/')
4. def home():
5.     return
   render_template('index.html')
6. if __name__ == '__main__':
7.     app.run(debug=True)
```

```
1. <!DOCTYPE html>
2. <html>
3. <head>
4.     <title>Website Flask</title>
5. </head>
6. <body>
7.     <h1>Selamat Datang</h1>
8.     <p>Website ini dibuat
   menggunakan Python dan
   Flask.</p>
9. </body>
10. </html>
```



Menambahkan CSS



Struktur folder saat ini:

ProyekFlask/

|

├─ app.py

├─ templates/

| └─ index.html

└─ static/

└─ style.css

```
1.  <!DOCTYPE html>
2.  <html>
3.  <head>
4.    <title>Website Flask</title>
5.    <link rel="stylesheet" href={{
      url_for('static', filename='style.css') }}>
6.  </head>
7.  <body>
8.    <h1>Smart Campus Dashboard</h1>
9.    <p>Sistem Monitoring IoT Kampus</p>
10. </body>
11. </html>
```

```
body {
  font-family: Arial, sans-serif;
  text-align: center;
  background-color: #f4f4f4;
}
h1 {
  color: darkblue;
}
```



Menampilkan Data



Struktur folder saat ini:

ProyekFlask/

|

└─ app.py

└─ templates/

| └─ index.html

└─ static/

└─ style.css

```
1. from flask import Flask, render_template
2. app = Flask(__name__)
3. @app.route('/')
4. def dashboard():
5.     data = {
6.         "energi": 1250,
7.         "air": 850,
8.         "sampah": 230
9.     }
10.    return
11.    render_template('index.html', data=data)
12.    app.run(debug=True)
```

```
1. <!DOCTYPE html>
2. <html>
3. <head>
4.     <title>Website Flask</title>
5.     <link rel="stylesheet" href="{{ url_for('static',
6. filename='style.css') }}">
7. </head>
8. <body>
9.     <h1>Smart Campus Dashboard</h1>
10.    <p>Sistem Monitoring IoT Kampus</p>
11.    <ul>
12.        <li>Energi : {{ data.energi }} kWh</li>
13.        <li>Air : {{ data.air }} Liter</li>
14.        <li>Sampah : {{ data.sampah }} Kg</li>
15.    </ul>
16. </body>
17. </html>
```



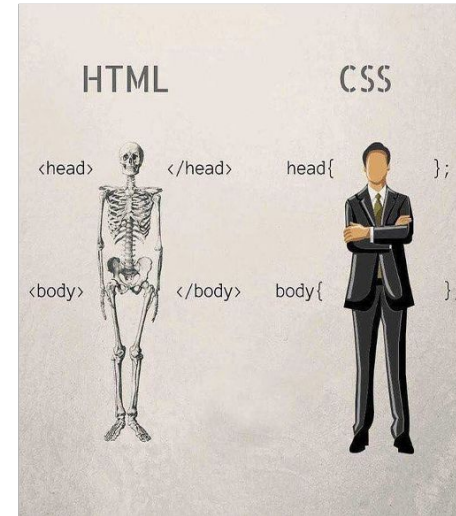
HTML

Hypertext Markup Language

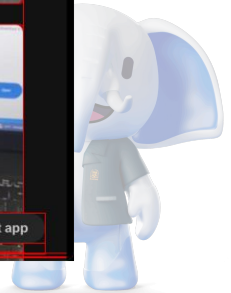
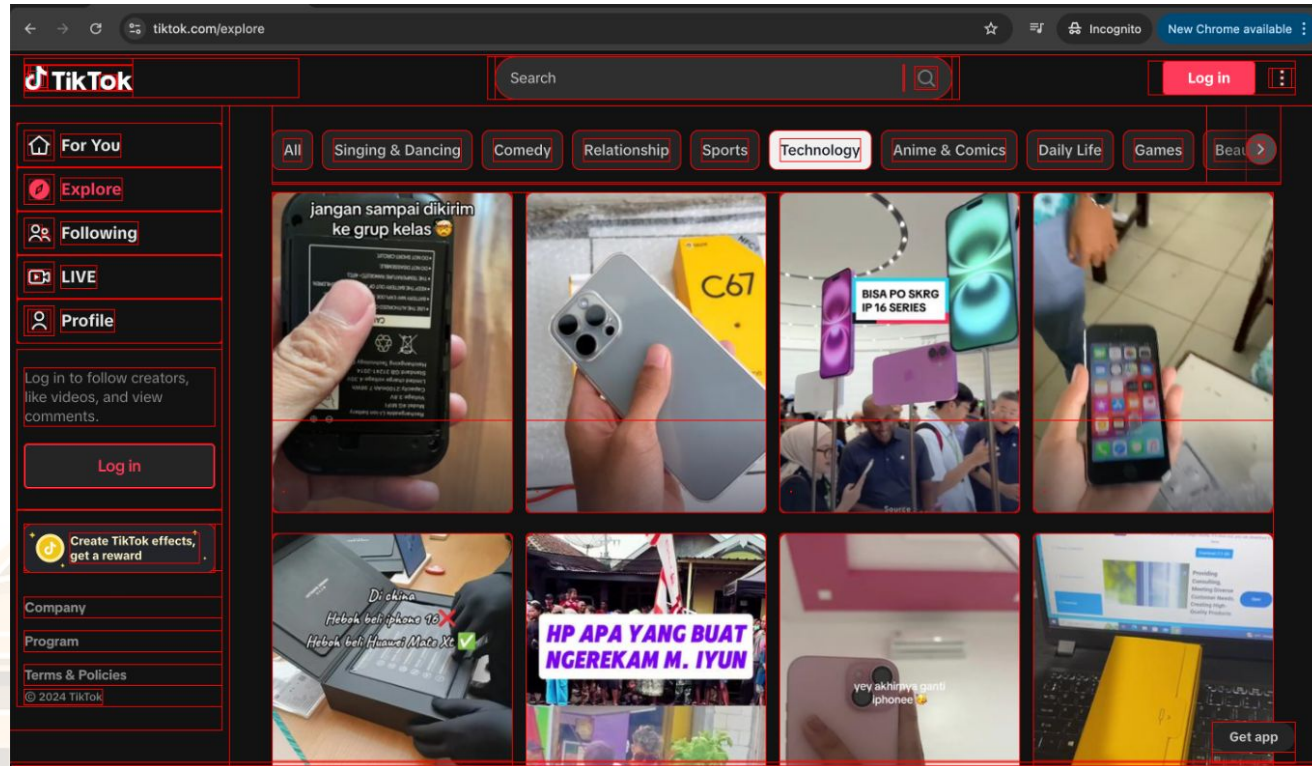


What is HTML?

HTML is a markup language used for structuring and presenting content on the World Wide Web.

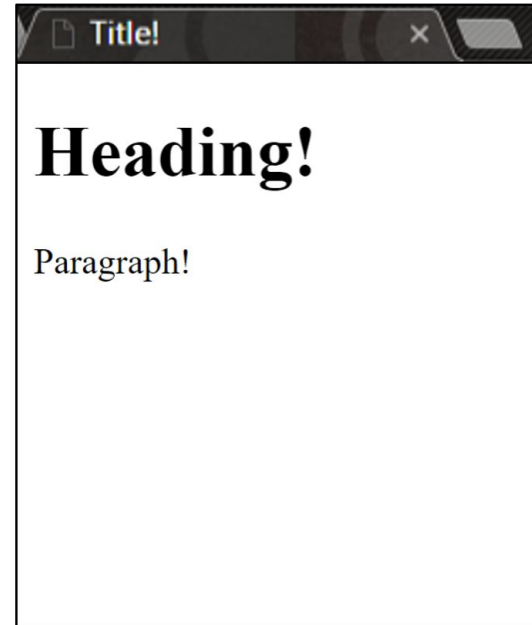


HTML = Nested Boxes

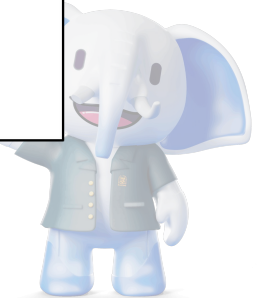


HTML Document Structure

```
hello.html  this makes sure page uses latest html and  
            not some random fallback version  
  
    <!DOCTYPE html>  
    <html>  
      <head>  
        <title>Title!</title>  
      </head>  
      <body>  
        <h1>Heading!</h1>  
        <p>Paragraph!</p>  
      </body>  
    </html>
```



Note: <head> primarily holds metadata information for machine processing, not human-readability. For human-visible information, like top-level headings and listed authors, see the [<header>](#) element.



Anatomy of HTML Element

attribute value

Opening tag

`<elname foo="bar">`

content goes here

`</elname>`

Closing tag

Elements and [tags](#) are *not* the same things. Tags begin or end an element in source code, whereas elements are part of the [DOM](#), the document model for displaying the page in the [browser](#).



Basic HTML Element

html

Root of HTML Document

head

Info about Document

body

Document Body

h1, h2, h3, ...

Header

p

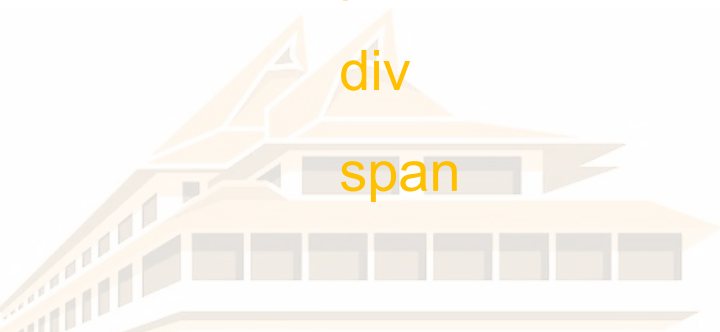
Paragraph

div

Generic block section

span

Generic inline section



Block vs Inline Element

BLOCK-LEVEL ELEMENTS:



INLINE ELEMENTS:



Block Level Elements

A block-level element always starts on a new line, and the browsers automatically add some space (a margin) before and after the element.

A block-level element always takes up the full width available (stretches out to the left and right as far as it can).

Two commonly used block elements are: `<p>` and `<div>`.

Inline Elements

An inline element does not start on a new line.

An inline element only takes up as much width as necessary.

Note: An inline element cannot contain a block-level element!



div & span

<div> Block Section in Document

**** Inline Section in Document

```
<div id="div-0">
  <h2>Div 1!</h2>
  <p>Paragraph<p>
</div>

<div id="div-1">
  <h2>Div 2!</h2>
  <p>Paragraph!!</p>
</div>

<span id="span-0">This text is inside a span element.</span>
<span id="span-1" span>This text is inside another span element.</span>
```

Div 1!

Paragraph

Div 2!

Paragraph!!

This text is inside a span element. This text is inside another span element.

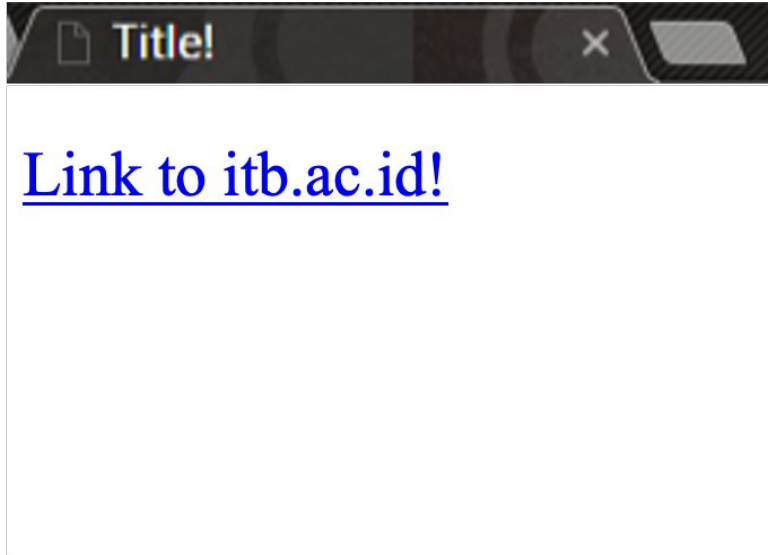


Example: Inserting Link

```
<a href="https://itb.ac.id">
```

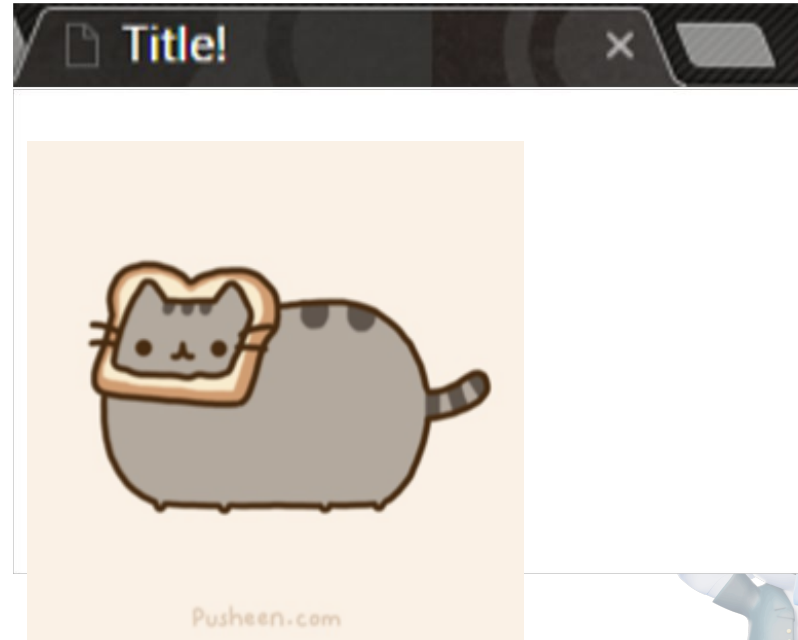
Link to itb.ac.id!

```
</a>
```



Example: Inserting Image

`</img>` ❌
``



Example: Inserting Lists

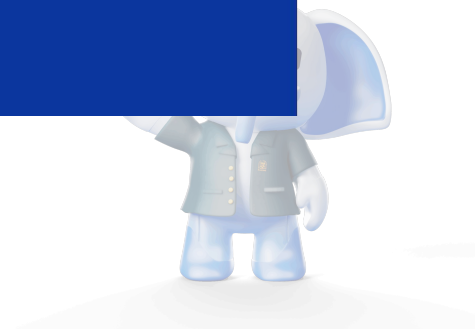
- `<ol>` Ordered List (1, 2, 3...)
- `<ul>` Unordered List (bullets)
- `<li>` List Item

hello.html

```
<!DOCTYPE html>
<html>
  <head>
    <title>Title!</title>
  </head>
  <body>
    <ul>
      <li>Item 1</li>
      <li>Item 2</li>
    </ul>
  </body>
</html>
```

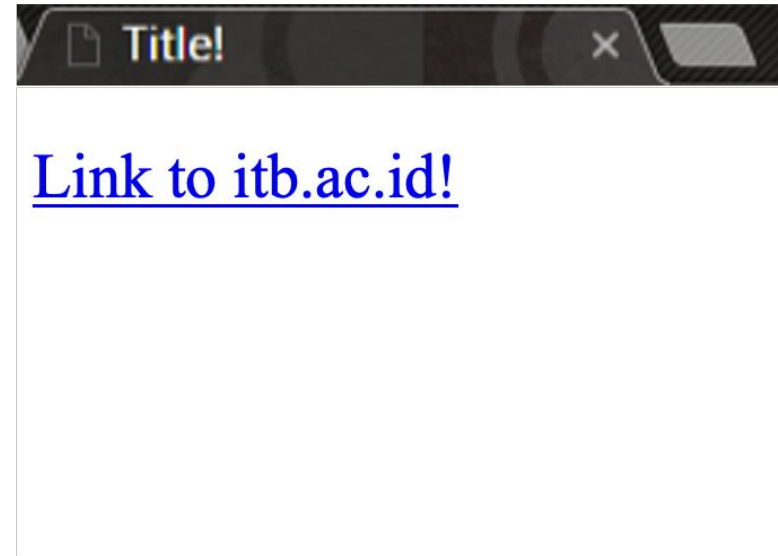
Title!

- Item 1
- Item 2



Example: Comment

```
<!-- Write your comments here  
-->  
<a href="https://itb.ac.id">  
Link to itb.ac.id!  
</a>
```



So many elements....

HTML elements reference

<https://developer.mozilla.org/en-US/docs/Web/HTML/Element>



HTML 5

HTML5 is the fifth and last major HTML version that is a World Wide Web Consortium (W3C) recommendation.

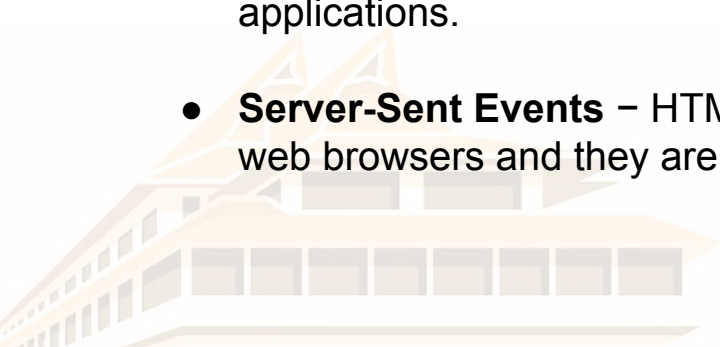
HTML5 is the next major revision of the HTML standard superseding HTML 4.01, XHTML 1.0, and XHTML 1.1.

HTML5 standard is a cooperation between the World Wide Web Consortium (W3C) and the Web Hypertext Application Technology Working Group (WHATWG).



HTML 5 New Features [1]

- **New Semantic Elements** – These are like <header>, <footer>, and <section>.
- **Forms 2.0** – Improvements to HTML web forms where new attributes have been introduced for <input> tag.
- **Persistent Local Storage** – To achieve persistence locally without resorting to third-party plugins.
- **WebSocket** – A next-generation bidirectional communication technology for web applications.
- **Server-Sent Events** – HTML5 introduces events which flow from web server to the web browsers and they are called Server-Sent Events (SSE).



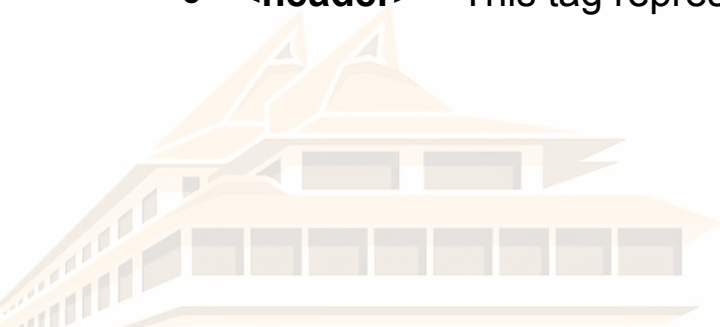
HTML 5 New Features [2]

- **Canvas** – This supports a two-dimensional drawing surface (bitmap) that you can program (manipulate) with JavaScript.
- **Audio & Video** – You can embed audio or video on your webpages without resorting to third-party plugins.
- **Geolocation** – Now visitors can choose to share their physical location with your web application.
- **Microdata** – This lets you create your own vocabularies beyond HTML5 and extend your web pages with custom semantics.
- **Drag and drop** – Drag and drop the items from one location to another location on the same webpage.



HTML 5 Semantic Elements [1]

- **<section>** – This tag represents a generic document or application section. It can be used together with h1-h6 to indicate the document structure.
- **<article>** – This tag represents an independent piece of content of a document, such as a blog entry or newspaper article.
- **<aside>** – This tag represents a piece of content that is only slightly related to the rest of the page.
- **<header>** – This tag represents the header of a section.



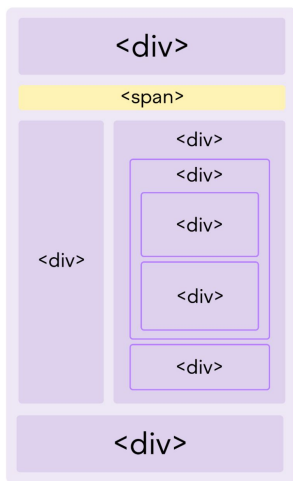
HTML 5 Semantic Elements [2]

- **<footer>** – This tag represents a footer for a section and can contain information about the author, copyright information, et cetera.
- **<nav>** – This tag represents a section of the document intended for navigation.
- **<dialog>** – This tag can be used to mark up a conversation.
- **<figure>** – This tag can be used to associate a caption together with some embedded content, such as a graphic or video.

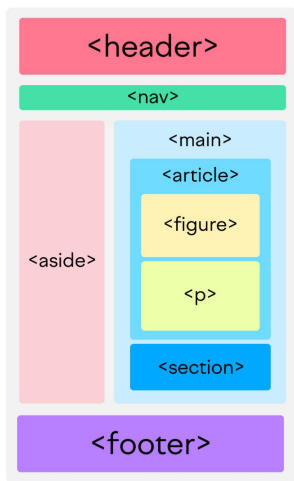


Non-Semantic vs Semantic HTML

Non-Semantic HTML



Semantic HTML



Some of the benefits from writing semantic markup are as follows:

- Search engines will consider its contents as important keywords to influence the page's search rankings (see [SEO](#))
- Screen readers can use it as a signpost to help visually impaired users navigate a page
- Finding blocks of meaningful code is significantly easier than searching through endless `divs` with or without semantic or namespaced classes
- Suggests to the developer the type of data that will be populated
- Semantic naming mirrors proper custom element/component naming



Web Forms 2.0

Web Forms 2.0 is an extension to the forms features found in HTML4.

Form elements and attributes in HTML5 provide a greater degree of semantic markup than HTML4 and free us from a great deal of tedious scripting and styling that was required in HTML4.



The <input> types element in HTML5

- **datetime** - a date and time (year, month, day, hour, minute, second, fractions of a second) encoded according to ISO 8601 with the time zone set to UTC
- **datetime-local** - a date and time with no time zone information
- **date** - a date (year, month, day)
- **month** - a date consisting of a year and a month
- **week** - a date consisting of a year and a week number
- **time** - a time (hour, minute, seconds, fractional seconds)
- **number** - accepts only numerical value. The step attribute specifies the precision, defaulting to 1
- **range** - used for input fields that should contain a value from a range of numbers
- **email** - It accepts only email value. This type is used for input fields that should contain an e-mail address.
- **url** - It accepts only URL value. This type is used for input fields that should contain a URL address.



Explore Other Cool Features...

HTML Living Standard:

<https://html.spec.whatwg.org/>



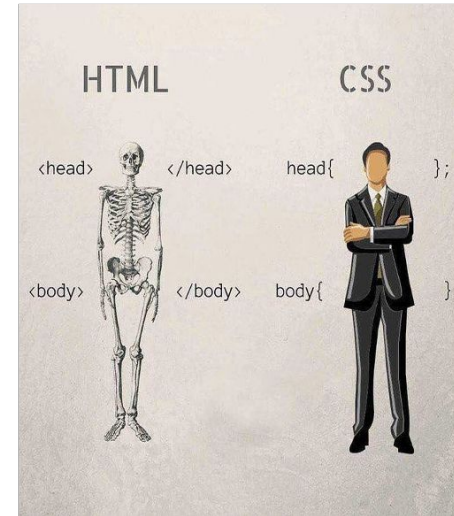
CSS

Cascading Style Sheets



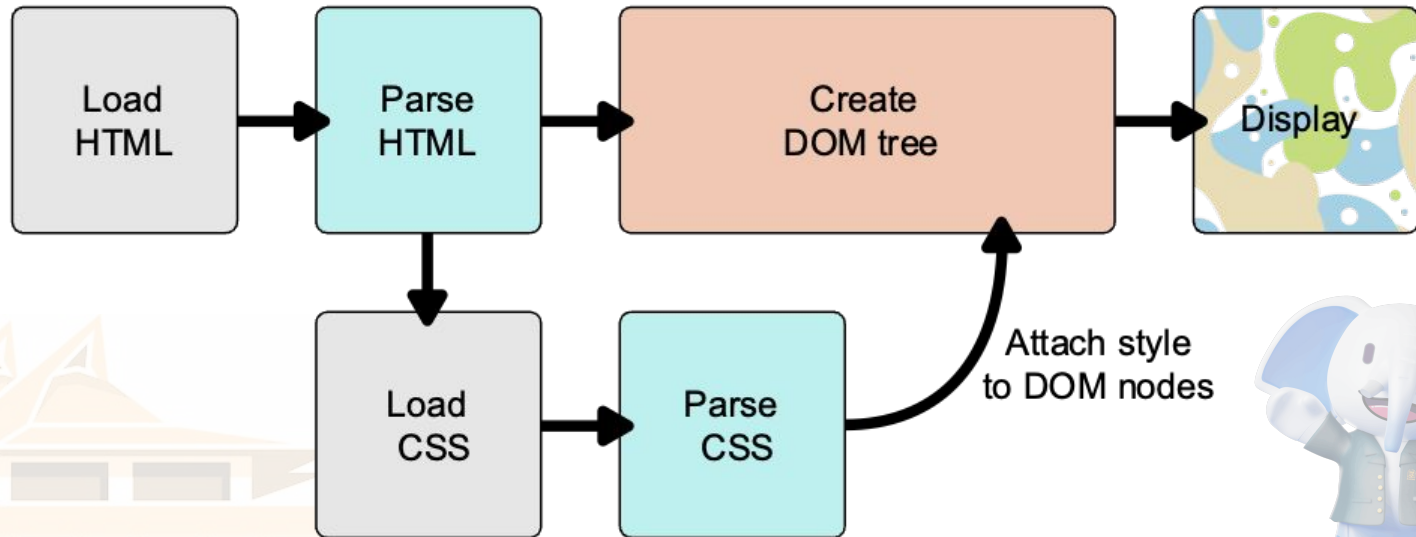
What is CSS?

A stylesheet language used to describe the presentation of a document written in HTML or XML (including XML dialects such as SVG, MathML or XHTML).



How does CSS actually work?

When a browser displays a webpage, it merges the content with its styling information. The browser goes through several key steps, and different browsers may handle the process slightly differently. The following diagram provides a simplified illustration of the process.



CSS & DOM

- Each element, attribute, and piece of text in the markup language becomes a DOM node in the tree structure.
- When CSS is applied, the styles are associated with the DOM nodes

Example:

`<p>`

Let's use:

`<span>Cascading</span>`

`<span>Style</span>`

`<span>Sheets</span>`

`</p>`

```
P
├ "Let's use:"
├ SPAN
│   └ "Cascading"
├ SPAN
│   └ "Style"
└ SPAN
    └ "Sheets"
```



Applying CSS to the DOM

Example:

```
<p>
```

Let's use:

```
<span>Cascading</span>
```

```
<span>Style</span>
```

```
<span>Sheets</span>
```

```
</p>
```

```
span {
```

```
  border: 1px solid black;
```

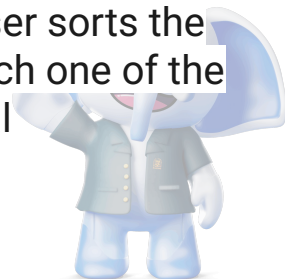
```
  background-color: lime;
```

```
}
```

Result:

Let's use: Cascading Style Sheets

The browser parses the HTML and creates a DOM from it. Next, it parses the CSS. Since the only rule available in the CSS has a span selector, the browser sorts the CSS very quickly! It applies that rule to each one of the three `<span>`s, then paints the final visual representation to the screen.



The CSS Cascade

The cascade is the algorithm for solving conflicts where multiple CSS rules apply to an HTML element.

```
button {  
  color: red;  
}
```

```
button {  
  color: blue;  
}
```



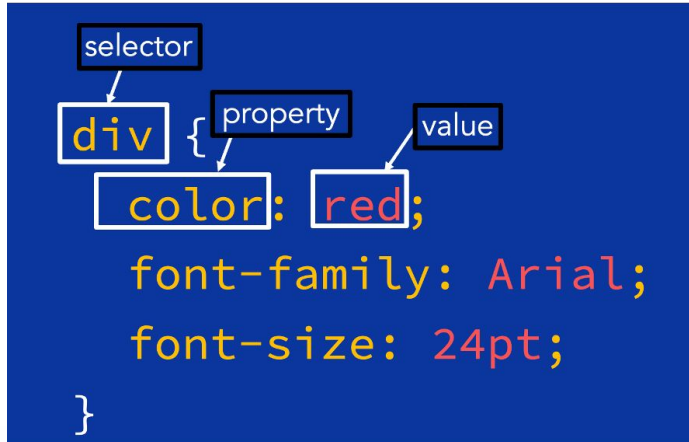
Hello, I have blue text

The cascade algorithm is split into 4 distinct stages.

1. **Position and order of appearance:** the order of which your CSS rules appear
2. **Specificity:** an algorithm which determines which CSS selector has the strongest match
3. **Origin:** the order of when CSS appears and where it comes from, whether that is a browser style, CSS from a browser extension, or your authored CSS
4. **Importance:** some CSS rules are weighted more heavily than others, especially with the !important rule type



CSS ruleset (rule)



In CSS, selectors are used to target the HTML elements on our web pages that we want to style. There are a wide variety of CSS selectors available, allowing for fine-grained precision when selecting elements to style.

List of selectors, properties :

<https://developer.mozilla.org/en-US/docs/Web/CSS/Reference>



Using Type Selector

hello.html

```
<h1>Heading!</h1>
<div>Paragraph!</div>
<div>Info</div>
```

style.css

```
div {
  color: red;
  font-family: Arial;
  font-size: 24pt;
}
```

Title!

Heading
Paragraph!
Info

Type selectors target an HTML element such as a `<div>`



Using Class Selector

hello.html

```
<h1>Heading!</h1>
<div>Paragraph!</div>
<div class="info">Info</div>
```

style.css

```
.info {
  color: red;
  font-family: Arial;
  font-size: 24pt;
}
```

Title!

Heading
Paragraph!
Info

Class selectors target an element that has a specific value for its class attribute.



Using ID Selector

hello.html

```
<h1>Heading!</h1>
<div>Paragraph!</div>
<div id="unique">Info</div>
```

style.css

```
#unique {
  color: red;
  font-family: Arial;
  font-size: 24pt;
}
```

ID selectors target an element that has a specific value for its id attribute.



ID vs Class Selector

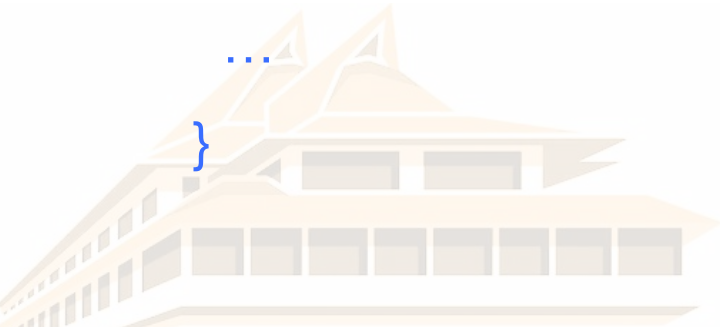
ID

- An element can have only one ID
- IDs must be unique in any given HTML document

```
#id {
```

```
...
```

```
}
```



Class

- An element can have multiple classes
- Can use the same class on multiple elements

```
.classname {
```

```
...
```

```
}
```



Using Attribute Selector

```
a[title] {  
}
```

```
a[href="https://example.com"]  
{  
}
```

Select elements based on the presence of a certain attribute or even the presence of an attribute with a particular value.



Pseudo-classes and pseudo-elements

This group of selectors includes pseudo-classes, which style certain states of an element.

```
a:hover {  
}
```

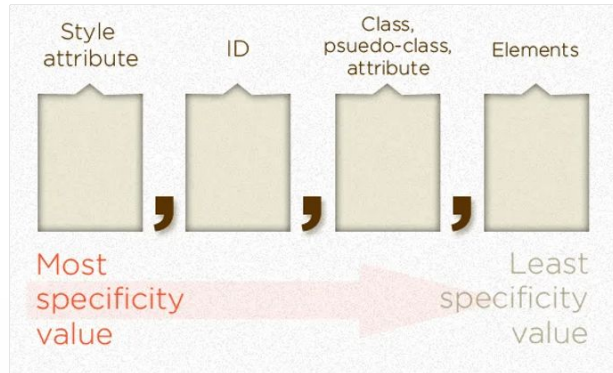
Select a certain part of an element rather than the element itself.

```
p::first-line {  
}
```



CSS Specificity

Think of specificity as a score/rank that determines which style declaration is ultimately applied to an element.



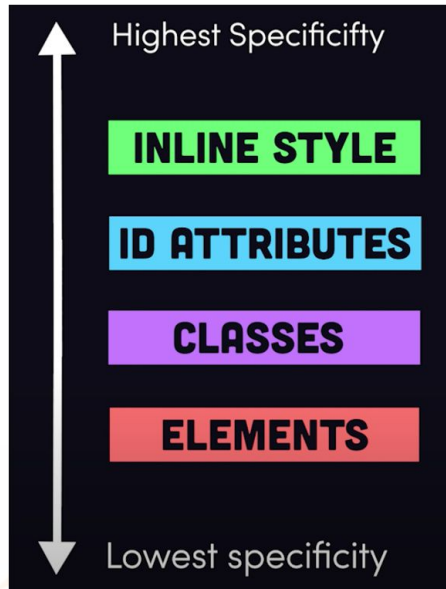
Start at 0, add 100 for each ID value, add 10 for each class value (or pseudo-class or attribute selector), add 1 for each element selector or pseudo-element.

Note 1: Inline style gets a specificity value of 1000, and is always given the highest priority!

Note 2: There is one exception to this rule: if you use the [!important](#) rule, it will even override inline styles!



CSS Hierarchy



Tips!

Only use classes for CSS styling!



CSS !important

- The **!important** rule in CSS is used to add more importance to a property/value than normal. In fact, if you use the !important rule, it will **override ALL previous styling rules** for that specific property on that element!
- The only way to override an !important rule is to include another !important rule on a declaration with the same (or higher) specificity in the source code



Example: CSS !important

```
#myid {  
  background-color: blue;  
}  
  
.myclass {  
  background-color: gray;  
}  
  
p {  
  background-color: red !important;  
}
```

```
<body>  
  <p>Hello World!</p>  
  <p class="myclass">Hello World!</p>  
  <p id="myid" >Hello World!</p>  
</body>
```

```
#myid {  
  background-color: blue; !important;  
}  
  
.myclass {  
  background-color: gray; !important;  
}  
  
p {  
  background-color: red !important;  
}
```

```
<body>  
  <p>Hello World!</p>  
  <p class="myclass">Hello World!</p>  
  <p id="myid" >Hello World!</p>  
</body>
```

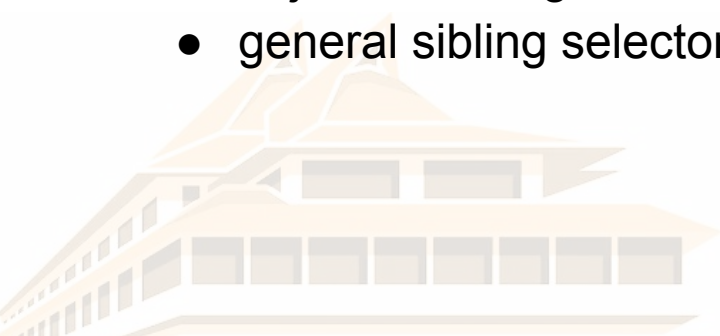


CSS Combinators

Specifies relationship between CSS selectors. Examples of selectors include HTML tags, such as **div** or **p**.

We have 4 different CSS combinators:

- descendant selector (space)
- child selector (>)
- adjacent sibling selector (+)
- general sibling selector (~)

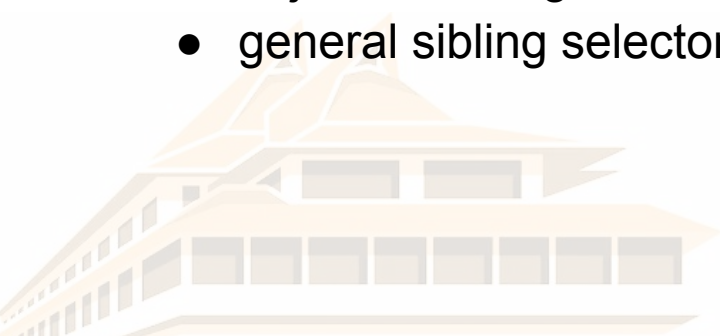


CSS Combinators

Specifies relationship between CSS selectors. Examples of selectors include HTML tags, such as **div** or **p**.

We have 4 different CSS combinators:

- descendant selector (space)
- child selector (>)
- adjacent sibling selector (+)
- general sibling selector (~)



HTML Example

Take a look at an example skeleton in the HTML section, followed by some code in the CSS section.

```
HTML

<div class="container">
  <section class="child" id="c1">
    <div>subchild 1</div>
  </section>
  <div class="child" id="c2">child 2</div>
  <div class="child" id="c3">child 3</div>
  <div class="child" id="c4">child 4</div>
</div>
```

```
CSS

.container {

}
```



Descendant selector (space)

Matches all elements that are descendants of the specified element

In the below example, we're selecting all the **div** elements that are descendants of the **.container** element → **can be deeply nested** (subchild 1 is also selected)



```
.container div {  
  font-size: 20px;  
}
```

subchild 1

child 2

child 3

child 4



Child selector (>)

Matches all elements that are **direct children of the specified element**

-In the below example, only the elements denoted as **child** will be selected → further levels of nesting will not be selected

```
● ● ● CSS
.container > div {
  font-size: 20px;
}
```

subchild 1

child 2

child 3

child 4



Adjacent Sibling Selector (+)

Selects a *single* element that is **directly after another specific element**

-In the below example, the **element directly after the element with an id of “c1”** is selected

```
CSS

#c1 + div {
  color: red;
}
```

subchild 1

child 2

child 3

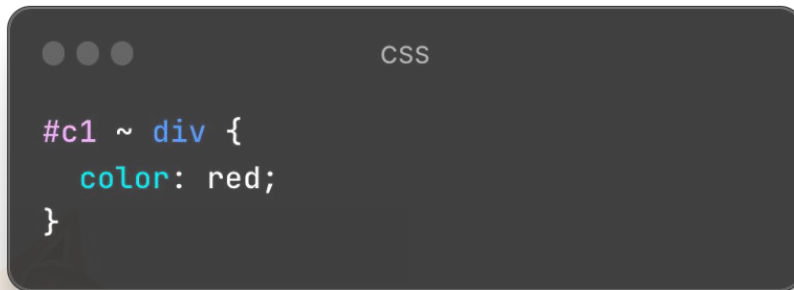
child 4



General Sibling Selector (~)

Selects *all* elements **after another specific element**

-In the below example, **all elements after the element with an id of “c1”** are selected



```
#c1 ~ div {  
  color: red;  
}
```

subchild 1

child 2

child 3

child 4



CSS Layout

CSS page layout techniques enable precise control over the positioning of elements within a webpage. These elements can be positioned based on several factors, including their default position in the normal layout flow, their relation to surrounding elements, their parent container, and the main viewport or window

Some CSS layout techniques:

Normal flow

The display property

Flexbox

Grid

Floats

Positioning

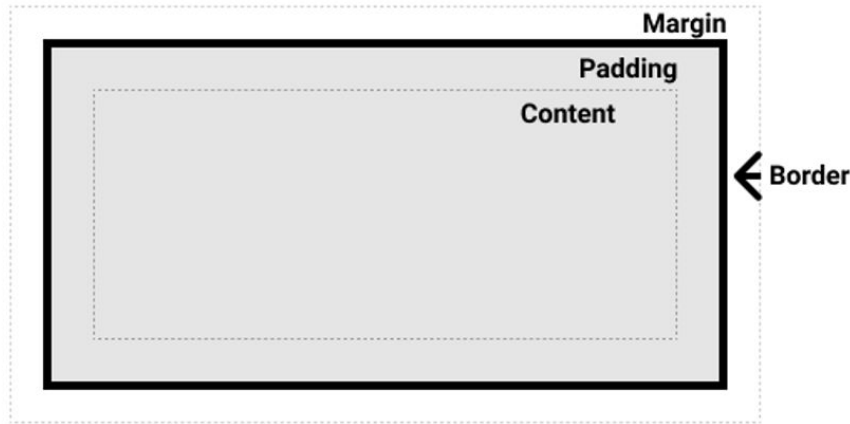
Table layout

Multiple-column layout



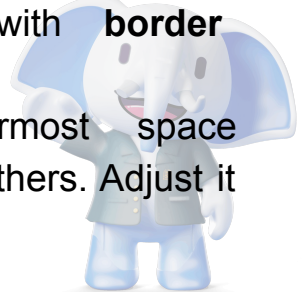
CSS Box Model

Everything in CSS has a box around it, and understanding these boxes is key to being able to create more complex layouts with CSS, or to align items with other items.



Parts of a box in CSS:

- **Content box:** The area where the content is displayed. You can size it with **width**, **height**, or similar properties.
- **Padding box:** The space around the content. Set its size using **padding**.
- **Border box:** The layer around the content and padding. Size it with **border** properties.
- **Margin box:** The outermost space between the element and others. Adjust it using **margin**.



Normal Flow

Normal flow is how the browser lays out HTML pages by default when you do nothing to control page layout.

```
<p>I love my cat.</p>

<ul>
  <li>Buy cat food</li>
  <li>Exercise</li>
  <li>Cheer up friend</li>
</ul>

<p>The end!</p>
```

I love my cat.

- Buy cat food
- Exercise
- Cheer up friend

The end!



The display property

The main methods for achieving page layout in CSS all involve specifying values for the display property. This property allows us to change the default way something displays.

```
display: block;  
display: inline;  
display: inline-block;  
display: flex;  
display: inline-flex;  
display: grid;  
display: inline-grid;  
display: flow-root;
```

more....

More detail:

<https://developer.mozilla.org/en-US/docs/Web/CSS/display>



Flexbox

Flexbox is the short name for the Flexible Box Layout CSS module, designed to make it easy for us to lay things out in one dimension — either as a row or as a column. To use flexbox, you apply `display: flex` to the parent element of the elements you want to lay out; a

CSS

```
.wrapper {  
  display: flex;  
}
```

HTML

```
<div class="wrapper">  
  <div class="box1">One</div>  
  <div class="box2">Two</div>  
  <div class="box3">Three</div>  
</div>
```

Result:

One

Two

Three

Learn by playing a game:

<https://flexboxfroggy.com/>



Grid

While flexbox is designed for one-dimensional layout, Grid Layout is designed for two dimensions — lining things up in rows and columns.

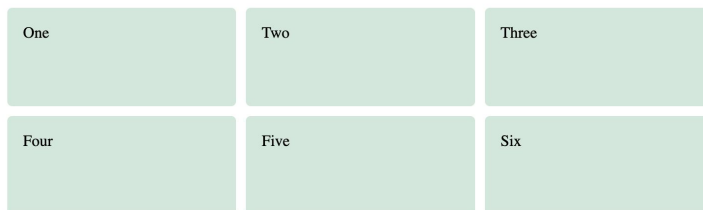
CSS

```
.wrapper {  
  display: grid;  
  grid-template-columns: 1fr 1fr 1fr;  
  grid-template-rows: 100px 100px;  
  gap: 10px;  
}
```

HTML

```
<div class="wrapper">  
  <div class="box1">One</div>  
  <div class="box2">Two</div>  
  <div class="box3">Three</div>  
  <div class="box4">Four</div>  
  <div class="box5">Five</div>  
  <div class="box6">Six</div>  
</div>
```

Result:



Learn by playing a game:
<https://cssgridgarden.com/>

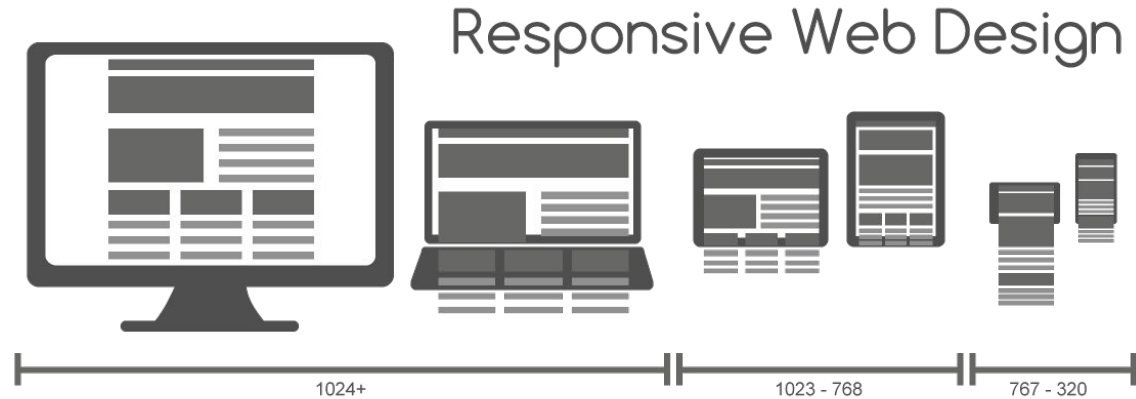


Flexbox vs Grid

Feature	Flexbox	Grid
Layout Dimension	One-dimensional (row <i>or</i> column)	Two-dimensional (rows <i>and</i> columns)
Primary Use Case	Aligning items in a line or list	Creating complex grid structures
Control	Control over space distribution along one axis	Full control over rows and columns
Item Placement	Items are placed in source order	Items can be placed freely within grid areas
Alignment	Great for aligning items along a single axis	Advanced alignment on both axes
Overlapping Elements	Not ideal	Supports overlap and layering
Browser Support	Wide support, older than Grid	Wide support, newer, but well-supported now



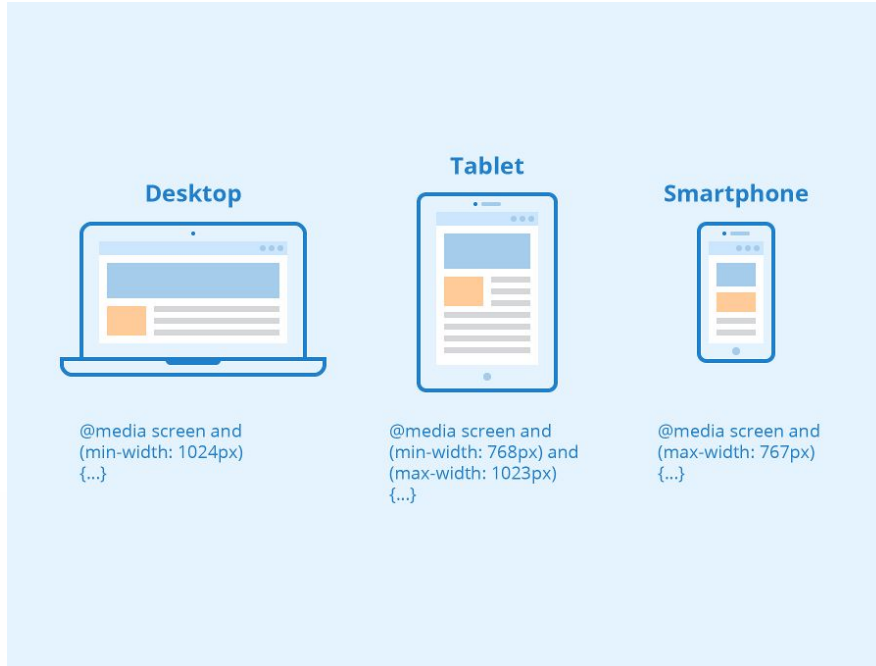
Responsive Web Design (RWD)



a web design approach to make web pages render well on all screen sizes and resolutions while ensuring good usability.



Media Queries



Media queries allow us to run a series of tests (e.g. whether the user's screen is greater than a certain width, or a certain resolution) and apply CSS selectively to style the page appropriately for the user's needs.



Responsive images/media

CSS

```
img,  
picture,  
video {  
    max-width: 100%;  
}
```

This scales media to ensure they never overflow their containers.

Using a single large image and scaling it down to fit small devices wastes bandwidth by downloading images larger than what is needed.



Responsive Images, using the [<picture>](#) element and the [](#) srcset and sizes attributes enables serving images targeted to the user's viewport and the device's resolution.



CSS Unit

px (absolute)

Avoid using `px` for font-sizes. Use mostly for small details like border and shadow.

- fixed in size
- not responsive
- overrides user's browser preferences

% (relative)

I recommend using percentages for layouts and width/height. For example, laying out links on navbar, placing images inside a div, etc.

- size is defined as percentage of another value (mostly parent element)
- sometimes size is defined as percentage of the element itself

em (relative)

You can use `em` for font-size and margin/padding. Use `em` when you want to adjust margin/padding based on that element's font-size (if font-size is big, you maybe want bigger spacing).

- changes behavior based on property.
- 1 em = parent font-size
- if parent doesn't have a size, defaults to 16px (body)

rem (relative)

You can also use `rem` for font-size and margin/padding. `rem` is easier to work with than `em` because it's more consistent.

- relative to root HTML, no matter what (default is 16px)
- you can change the root HTML size. For example, if you change it to 20px, 1 rem will always be 20px.

vw/vh (relative)

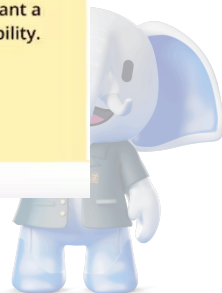
vw/vh are relative to the width/height of the browser window. 100vw means full width of the screen. Use vw/vh for bigger layouts, like background.

- useful for responsive websites because everything scales

ch

`ch` is relative to the width of the number 0 of the current font. `ch` is used to size the width of a paragraph. In general, you want a 45-70 character wide column for readability.

- You would use it like this:
max-width: 40ch;
This sets the width of the column to a maximum of 40 characters per line.



The viewport meta tag

This viewport meta tag tells mobile browsers that they should set the width of the viewport to the device width, and scale the document to 100% of its intended size, which shows the document at the mobile-optimized size that you intended.

HTML

```
<meta name="viewport" content="width=device-width,initial-scale=1" />
```



Without it, your responsive design with breakpoints and media queries may not work as intended on mobile browsers. So you should *always* include the viewport meta tag in the head of your documents.



Exercise

Buat halaman web portpfolio pribadi sebagai representasi digital dari karya, keterampilan, dan identitas Anda menggunakan HTML dan CSS dari awal (from scratch), tidak diperbolehkan menggunakan template atau CSS framework apapun atau Genarative AI.

Kriteria dan Tantangan Utama:

- 1. Gunakan Struktur HTML yang Semantic**
- 2. Desain Responsif**
- 3. Kreativitas dalam Penggunaan Warna dan Tipografi**

Bonus:

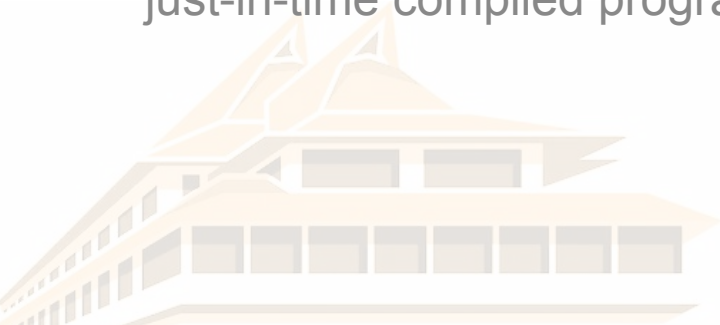
- 3. Desain yang Interaktif (without JavaScript, only HTML+CSS)**
- 5. Aksesibilitas Web**
- 6. Optimisasi Kinerja**

Deploy web anda pada penyediaan layanan hosting web statis (Netlify, Vercel, Static.app, Github Pages, etc...) Link dikumpulkan melalui Edunex kelas masing-masing berikan alasan atas keputusan desain yang ada ambil untuk kriteria-kriteria tersebut.



Java Script

just-in-time compiled programming language



What is JavaScript?

- A lightweight interpreted (or just-in-time compiled) programming language
- Most well-known as the scripting language for Web pages
- Many non-browser environments also use it, such as Node.js, Deno, etc.
- Formalized & standardised as ECMAScript (ECMA-262 and ISO/IEC 16262)



Brendan Eich

Created in 10 Days

Brendan Eich developed the first version of JavaScript in just 10 days in 1995.

Java ≠ JavaScript

Despite the name, Java and JavaScript are entirely different languages. The name similarity was a marketing strategy by Netscape and Sun Microsystems.



JavaScript in Your Browsers

When referring to JavaScript in browsers, it implies utilizing it for client-side scripting.

JavaScript in browsers extends the core language (specified in ECMAScript standard) by supplying objects to control a browser and its DOM (also known as the Web Platform API)

<https://developer.mozilla.org/en-US/docs/Web/API>

JS

```
const nameField = document.getElementById("userName");  
const sendButton = document.getElementById("sendButton");
```



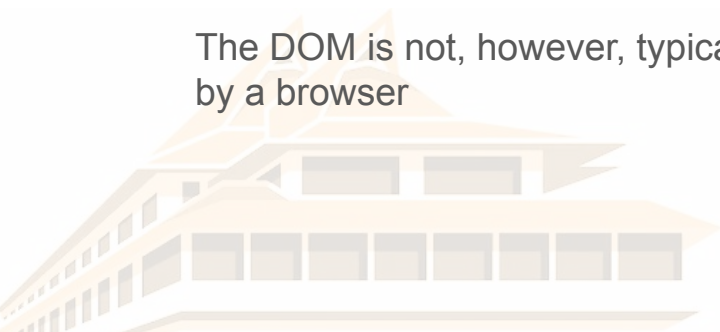
The JavaScript Engine

A special program embedded in a browser to execute JavaScript code. This program implements ECMAScript standard.

Has various implementation variant:

- V8 -> Chrome, Edge, Opera, **Node.js, Deno (not a browser)**
- SpiderMonkey -> Mozilla Firefox
- JavaScriptCore -> WebKit/Safari

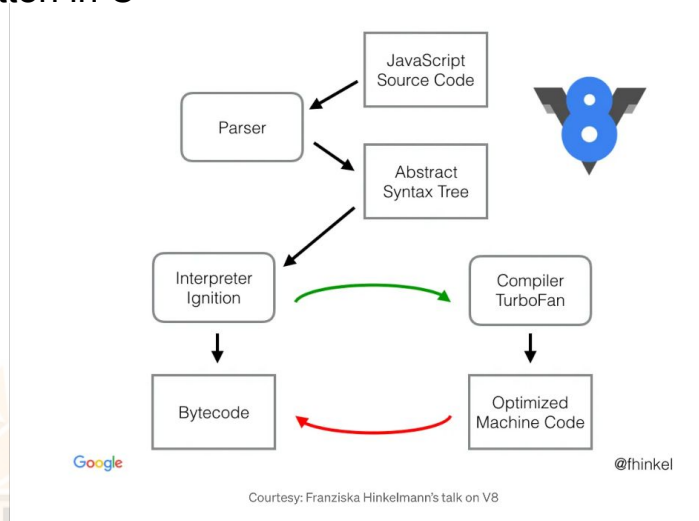
The DOM is not, however, typically provided by the JavaScript engine but instead by a browser



Inside the JavaScript Engine

V8 from Google is the most used JavaScript engine

It's written in C++



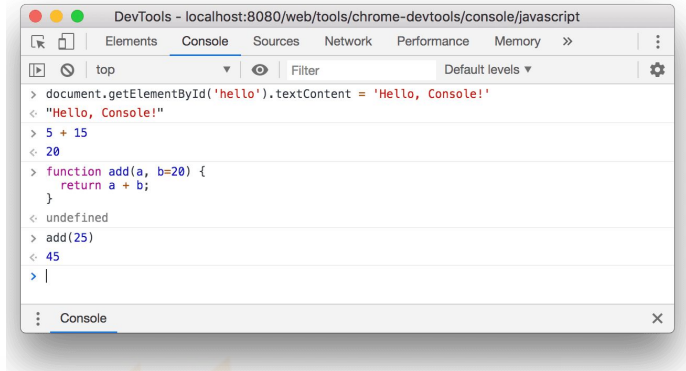
<https://www.youtube.com/watch?v=p-iiEDtpy6I>

V8 has had an interpreter as its first execution tier. This interpreter "compiles" JavaScript "just in time/JIT" to bytecode (which is then interpreted).



Where can we run JavaScript code?

The browser console



Tied to Our HTML files

For Example inside our index.html, just before `</body>`

HTML

```
<script src="scripts/main.js"></script>
```

scripts/main.js

JS

```
const myHeading = document.querySelector("h1");  
myHeading.textContent = "Hello world!";
```



JavaScript Basic



Basic Syntax

- Alike to Java or C
- Case-sensitive
- One line represents a statement
- “;” is optional to end a statment
- Comment: // and /* */



Data Types: Dynamic Typing

JavaScript is a dynamic language with dynamic types. Variables in JavaScript are not directly associated with any particular value type, and any variable can be assigned (and re-assigned) values of all types

JS

```
let foo = 42; // foo is now a number  
foo = "bar"; // foo is now a string  
foo = true; // foo is now a boolean
```



Data Types: Weak Typing

JavaScript is also a weakly typed language, which means it allows implicit type conversion when an operation involves mismatched types, instead of throwing type errors.

JS

```
const foo = 42; // foo is a number
const result = foo + "1"; // JavaScript coerces foo to a string, so it can be
concatenated with the other operand
console.log(result); // 421
```



Data Types: Primitive Types

In JavaScript, almost everything is object, unless the 7 primitive types (string, number, boolean, null, undefined, symbol in ES6, bigint in ES11).

Type	<code>typeof</code> return value	Object wrapper
Null	"object"	N/A
Undefined	"undefined"	N/A
Boolean	"boolean"	Boolean
Number	"number"	Number
BigInt	"bigint"	BigInt
String	"string"	String
Symbol	"symbol"	Symbol

More detail:

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Data_structures



Operator

JavaScript has both binary and unary operators, and one special ternary operator, the conditional operator. A binary operator requires two operands, one before the operator and one after the operator.

`operand1 operator operand2`

`operator operand`
`operand operator`

For example, `3 + 4` or `x * y`.

For example, `x++` or `++x`.

`condition ? exprIfTrue : exprIfFalse`



Operator Types

- [Assignment operators](#)
- [Comparison operators](#)
- [Arithmetic operators](#)
- [Bitwise operators](#)
- [Logical operators](#)
- [BigInt operators](#)
- [String operators](#)
- [Conditional \(ternary\) operator](#)
- [Comma operator](#)
- [Unary operators](#)
- [Relational operators](#)

a.

Operator	Example	Explanation	Output
+	5 + 3	Addition	8
-	5 - 2	Subtraction	3
*	4 * 2	Multiplication	8
/	10 / 2	Division	5
%	7 % 3	Modulus (Remainder)	1
**	2 ** 3	Exponentiation	8

b.

Operator	Example	Explanation	Output
==	5 == '5'	Checks equality (value only)	true
===	5 === '5'	Checks equality (value and type)	false
!=	5 != 4	Checks inequality	true
!==	5 !== '5'	Checks inequality (value and type)	true
>	7 > 3	Greater than	true
<	3 < 7	Less than	true
>=	5 >= 5	Greater than or equal to	true
<=	4 <= 5	Less than or equal to	true

For example: a. Arithmetic operator and b. Comparison operators

Defining variables

Variables are containers that store values. You start by declaring a variable with the let keyword, followed by the name you give to the variable:

```
let myBoolean = true;  
let myNumber = 12;  
let myString = "Hello World!";  
  
myBoolean = false;  
myNumber = -5.6;  
myString = "";
```

JavaScript convention
is to name variables
using **camelCase**.



Defining constants

To define a variable which cannot be re-assigned later:

```
const answerToLife = 6.148;  
  
// this WILL NOT work!!!  
answerToLife = 42;
```



let vs const

Why bother using `const` when `let` exists?

Safe code practices! If something should never be changed, don't let it change :)

```
const secondsPerMinute = 60;  
// if this needs to be changed, then  
// we have bigger issues to address
```



let vs var

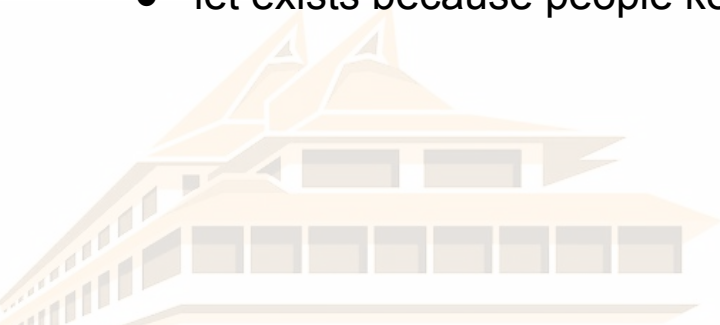
Short Answer: Please don't use var

```
let userLoggedIn = true;
```

```
var userLoggedIn = true;
```

Technical details (Google it if you're interested):

- let is block-scoped
- var is function-scoped
- let exists because people kept getting bugs when trying to use var



null vs undefined

undefined means "declared but not yet assigned a value"

null means "no value"

```
let firstName;  
// currently, firstName is undefined  
  
firstName = "Albert";  
// firstName has now been assigned to a value  
  
firstName = null;  
// we can explicitly "empty" the variable
```



Type coercion

Type coercion is the automatic or implicit conversion of values from one data type to another (such as strings to numbers). Type conversion is similar to type coercion because they both convert values from one data type to another with one key difference — type coercion is implicit whereas type conversion can be either implicit or explicit.

JS

```
const value1 = "5";  
const value2 = 9;  
let sum = value1 + value2;  
  
console.log(sum);
```



Type coercion

Type coercion is the automatic or implicit conversion of values from one data type to another (such as strings to numbers). Type conversion is similar to type coercion because they both convert values from one data type to another with one key difference — type coercion is implicit whereas type conversion can be either implicit or explicit.

JS

```
const value1 = "5";  
const value2 = 9;  
let sum = value1 + value2;  
  
console.log(sum);
```



Arrays [1]

For when you want to store a sequence of (ideally similar) items:

```
// initialize
let pets = ["cat", "dog", "guinea pig", "bird"];

// access
console.log(pets[3]); // "bird"

// replace
pets[2] = "hamster"; // ["cat", "dog", "hamster", "bird"]
```



Arrays [2]

For when you want to store a sequence of (ideally similar) items:

```
// initialize
let pets = ["cat", "dog", "guinea pig", "bird"];

// remove from end
pets.pop(); // ["cat", "dog", "guinea pig"]

// add to end
pets.push("rabbit"); // ["cat", "dog", "guinea pig", "rabbit"]
```

Check other array methods on your browser console!



Conditionals

We often want to perform different actions in response to different conditions.

For this, we use the **conditional operators** `if`, `else`, and `else if`:

```
if (hour < 12) {  
    console.log("Good morning!");  
} else if (hour < 16) {  
    console.log("Good afternoon!");  
} else if (hour < 20) {  
    console.log("Good evening!");  
} else {  
    console.log("Good night!");  
}
```



For loops

Useful when we want to iterate through indices:

```
const pets = ["cat", "dog", "guinea pig", "bird"];

for (let i = 0; i < pets.length; i++) {
  const phrase = "I love my " + pets[i];
  console.log(phrase);
}
```

I love my cat
I love my dog
I love my guinea pig
I love my bird



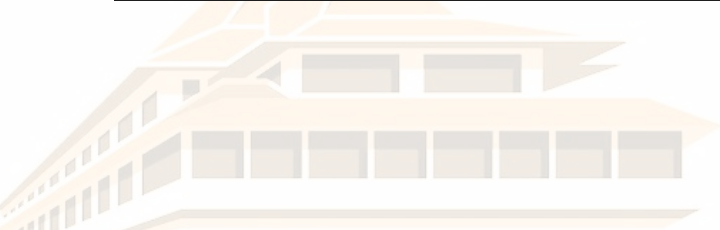
For ... of ...

A more “pythonic” way of iterating:

```
const pets = ["cat", "dog", "guinea pig", "bird"];

for (const animal of pets) {
  const phrase = "I love my " + animal;
  console.log(phrase);
}
```

I love my cat
I love my dog
I love my guinea pig
I love my bird



Objects

A JavaScript **object** is a collection of **name:value** pairs.

```
const myCar = {  
  make    : "Ford",  
  model   : "Mustang",  
  year    : 2005,  
  color   : "red"  
};
```



An object like this is referred to as **an object literal** — we've literally written out the object contents as we've come to create it.



Accessing Object Properties

There are two ways to access object properties, if you know the property name:

```
const myCar = {  
  make    : "Ford",  
  model   : "Mustang",  
  year    : 2005,  
  color   : "red"  
};  
  
console.log(myCar.model);    // "Mustang"  
console.log(myCar["color"]); // "red"
```



Object destructuring

Object destructuring is a shorthand to obtain multiple properties at once.

without object
destructuring

```
const myCar = {  
  make   : "Ford",  
  model  : "Mustang",  
  year   : 2005,  
  color  : "red"  
};  
  
const make = myCar.make;  
const model = myCar.model;
```

with object destructuring

```
const myCar = {  
  make   : "Ford",  
  model  : "Mustang",  
  year   : 2005,  
  color  : "red"  
};  
  
const { make, model } = myCar;
```



Object Equality...?

We use `===` to check if two *primitive* variables are equal in JavaScript.

```
2 === 2;           // true
2 === 3;           // false
"2" === "2";       // true
2 === "2";         // false
```

But what does `===` mean for arrays and objects?

```
let arr1 = [1, 2, 3];
let arr2 = [1, 2, 3];

arr1 === arr2; // false!
```

```
let person1 = { name: "Bill Gates" };
let person2 = { name: "Bill Gates" };

person1 === person2; // false!
```



Object References

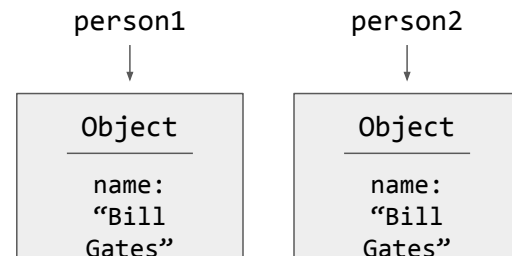
Object variables are **references** – they point to where the data is actually stored.

```
let person1 = { name: "Bill Gates" };  
let person2 = { name: "Bill Gates" };  
  
person1 === person2; // false!
```

=== checks if the *references* are equal.

Two objects created separately are stored separately, so their references are different!

Same goes for arrays – two arrays created separately have different references.



How to copy arrays and objects

It's not as simple as

```
let arr = [1, 2, 3];  
let copyArr = arr;
```

(Why not?)



One way to copy arrays and objects is to use the **spread** operator (`...`) like so:

```
let arr = [1, 2, 3];  
let copyArr = [...arr];
```

```
let obj = { name: "Bill Gates" };  
let copyObj = { ...obj };
```



How to copy arrays and objects

It's not as simple as

```
let arr = [1, 2, 3];  
let copyArr = arr;
```

(Why not?)



One way to copy arrays and objects is to use the **spread** operator (`...`) like so:

```
let arr = [1, 2, 3];  
let copyArr = [...arr];
```

```
let obj = { name: "Bill Gates" };  
let copyObj = { ...obj };
```



Functions

A function in JavaScript is similar to a procedure—a set of statements that performs a task or calculates a value, but for a procedure to qualify as a function, it should take some input and return an output where there is some obvious relationship between the input and the output.

JS

```
function square(number) {  
    return number * number;  
}
```

JS

```
() => expression  
  
param => expression  
  
(param) => expression  
  
(param1, paramN) => expression  
  
() => {  
    statements  
}  
  
param => {  
    statements  
}  
  
(param1, paramN) => {  
    statements  
}
```



Functions are first-class citizen

In JavaScript, functions are first-class objects, because they can be passed to other functions, returned from functions, and assigned to variables and properties. They can also have properties and methods just like any other object.

What distinguishes them from other objects is that functions can be called.

```
const greeting = function(name) {  
  return `Hello, ${name}!`;  
};  
console.log(greeting("Yuda")); // Output: Hello, Yuda!
```

```
function callWithTwo(fn) {  
  return fn(2);  
}  
  
const multiplyByTwo = function(num) {  
  return num * 2;  
};  
  
console.log(callWithTwo(multiplyByTwo)); // Output: 4
```



Function hoisting

This code runs without any error, despite the `square()` function being called before it's declared.

```
console.log(square(5)); // 25

function square(n) {
  return n * n;
}
```

↓
the JavaScript interpreter hoists the entire **function declaration to the top** of the current scope



Function Parameter

Default Parameter Value

```
function multiply(a, b = 1) {  
  return a * b;  
}  
  
console.log(multiply(5)); // 5
```

Rest parameters

```
function multiply(multiplier, ...theArgs) {  
  return theArgs.map((x) => multiplier * x);  
}  
  
const arr = multiply(2, 1, 2, 3);  
console.log(arr); // [2, 4, 6]
```



OOP and JavaScript : Constructor

We would like a way to define the set of methods and the properties it can have like class in other programming languages. Then, we can create as many objects as we like from it.

```
function Person(name) {  
  this.name = name;  
  this.introduceSelf = function () {  
    console.log(`Hi! I'm ${this.name}.`);  
  };  
}
```

Constructor is a function start with a capital letter.

```
const salva = new Person("Salva");  
salva.introduceSelf();  
// "Hi! I'm Salva."  
  
const frankie = new Person("Frankie");  
frankie.introduceSelf();  
// "Hi! I'm Frankie."
```

We use **new** keyword to call Person() as a constructor.



OOP and JavaScript : Prototype Chain

Prototypes are the mechanism by which JavaScript objects inherit features from one another (a version of inheritance)

```
const personPrototype = {  
  greet() {  
    console.log(`hello, my name is ${this.name}!`);  
  },  
};  
  
function Person(name) {  
  this.name = name;  
}  
  
Object.assign(Person.prototype, personPrototype);  
// or  
// Person.prototype.greet = personPrototype.greet;
```

More detail:

https://developer.mozilla.org/en-US/docs/Learn/JavaScript/Objects/Object_prototypes

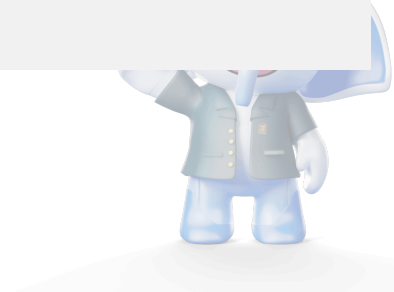


JavaScript Classes (ES6++)

JavaScript classes are essentially just syntactic sugar over constructor functions and prototypes.

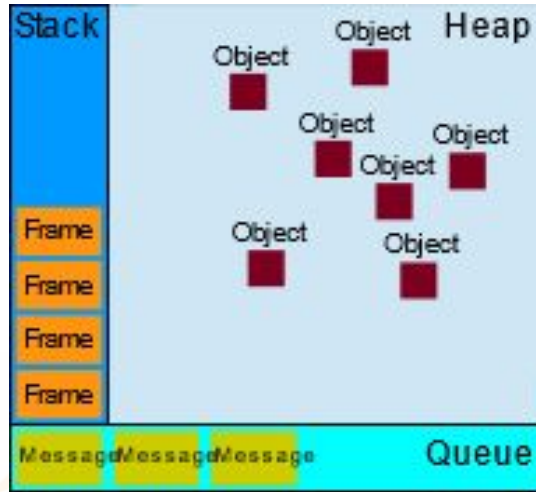
```
class Animal {  
  constructor(name) {  
    this.name = name;  
  }  
  
  speak() {  
    console.log(`${this.name} makes a noise.`);  
  }  
}
```

```
class Dog extends Animal {  
  constructor(name) {  
    super(name); // call the super class constructor and pass in the name parameter  
  }  
  
  speak() {  
    console.log(`${this.name} barks.`);  
  }  
}  
  
const d = new Dog("Mitzie");  
d.speak(); // Mitzie barks.
```



Asynchronous JavaScript

When multiple related things happen without any being dependent on the completion of previous happenings, they are asynchronous.



JavaScript program is *single-threaded*. A thread is a sequence of instructions that a program follows. Because the program consists of a single thread, it can only do one thing at a time: **so if it is waiting for our long-running synchronous call to return, it can't do anything else.**



JavaScript has a runtime model based on an **event loop**:
<https://www.youtube.com/watch?v=eiC58R16hb8>

Async JS: Event Handler

You provide a function (the event handler) that will be called, not right away, but whenever the event happens

Example using
asynchronous API:
XMLHttpRequest

```
const log = document.querySelector(".event-log");

document.querySelector("#xhr").addEventListener("click", () => {
  log.textContent = "";

  const xhr = new XMLHttpRequest();

  xhr.addEventListener("loadend", () => {
    log.textContent = `${log.textContent}Finished with status: ${xhr.status}`;
  });

  xhr.open(
    "GET",
    "https://raw.githubusercontent.com/mdn/content/main/files/en-us/_wikihistory.json",
  );
  xhr.send();
  log.textContent = `${log.textContent}Started XHR request\n`;
});
```



Async JS: Callback

A callback is just a function that's passed into another function, with the expectation that the callback will be called at the appropriate time.

Callback-based code can get hard to understand when the callback itself has to call functions that accept a callback

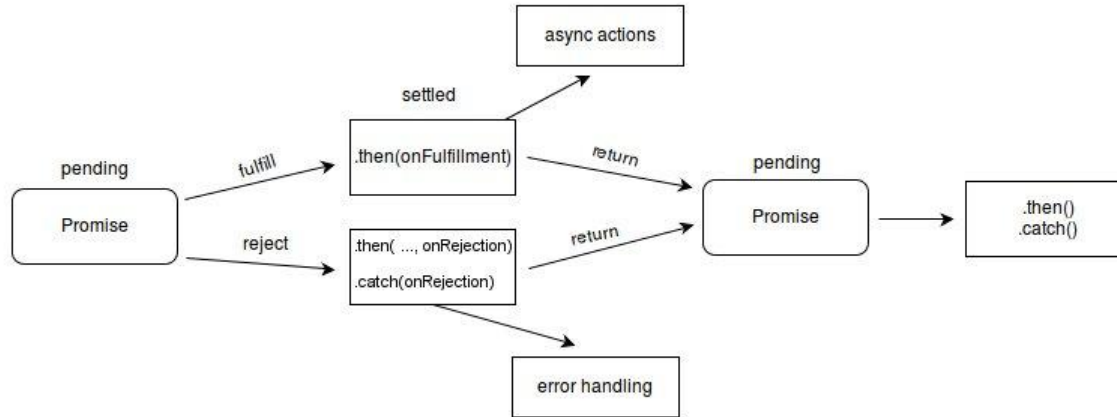
This is sometimes called "callback hell" or the "pyramid of doom".

```
function doStep1(init, callback) {  
  const result = init + 1;  
  callback(result);  
}  
  
function doStep2(init, callback) {  
  const result = init + 2;  
  callback(result);  
}  
  
function doStep3(init, callback) {  
  const result = init + 3;  
  callback(result);  
}  
  
function doOperation() {  
  doStep1(0, (result1) => {  
    doStep2(result1, (result2) => {  
      doStep3(result2, (result3) => {  
        console.log(`result: ${result3}`);  
      });  
    });  
  });  
}  
  
doOperation();
```



Async JS: Promise

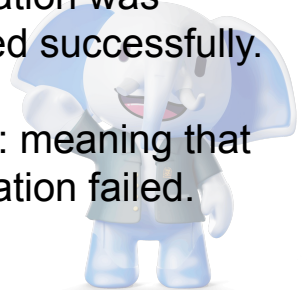
The Promise object represents the eventual completion (or failure) of an asynchronous operation and its resulting value. It allows you to associate handlers with an asynchronous action's eventual success value or failure reason



pending: initial state, neither fulfilled nor rejected.

... **fulfilled:** meaning that the operation was completed successfully.

rejected: meaning that the operation failed.



Async JS: Promise Fetch Example

```
// Make a request for user.json
fetch('/article/promise-chaining/user.json')
  // Load it as json
  .then(response => response.json())
  // Make a request to GitHub
  .then(user => fetch(`https://api.github.com/users/${user.name}`))
  // Load the response as json
  .then(response => response.json())
  // Show the avatar image (githubUser.avatar_url) for 3 seconds (maybe)
  .then(githubUser => {
    let img = document.createElement('img');
    img.src = githubUser.avatar_url;
    img.className = "promise-avatar-example";
    document.body.append(img);

    setTimeout(() => img.remove(), 3000); // (*)
  });
```

Fetch is the modern replacement for XMLHttpRequest: unlike XMLHttpRequest (XHR), which uses callbacks, Fetch is promise-based.

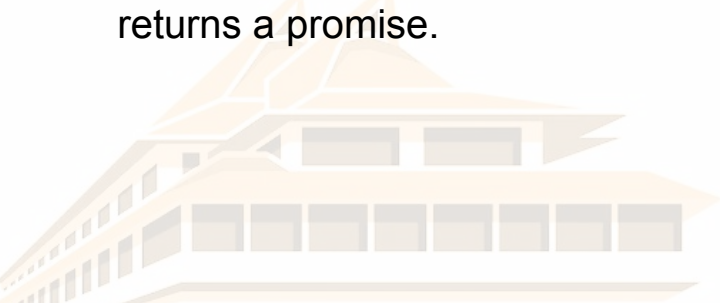


Async JS: Async Await

The `async` keyword gives you a simpler way to work with asynchronous promise-based code. Adding `async` at the start of a function makes it an async function:

Inside an async function, you can use the `await` keyword before a call to a function that returns a promise.

```
async function fetchProducts() {  
  try {  
    // after this line, our function will wait for the `fetch()` call to be settled  
    // the `fetch()` call will either return a Response or throw an error  
    const response = await fetch(  
      "https://mdn.github.io/learning-area/javascript/apis/fetching-data/can-store/products.json",  
    );  
    if (!response.ok) {  
      throw new Error(`HTTP error: ${response.status}`);  
    }  
    // after this line, our function will wait for the `response.json()` call to be settled  
    // the `response.json()` call will either return the parsed JSON object or throw an error  
    const data = await response.json();  
    console.log(data[0].name);  
  } catch (error) {  
    console.error(`Could not get products: ${error}`);  
  }  
}  
  
fetchProducts();
```



TypeScript Basic

 TypeScript

Made with ♥ in Redmond,
Boston, SF & Dublin

 Microsoft

© 2012-2024 Microsoft



Typescript is statically typed

Type 'undefined' is not assignable to type 'number'. ts(2322)

[View Problem \(\F8\)](#) No quick fixes available

```
let five: number = 5;  
five = undefined;  
console.log(five);
```

Cannot run code without
fixing error!



TypeScript, formally

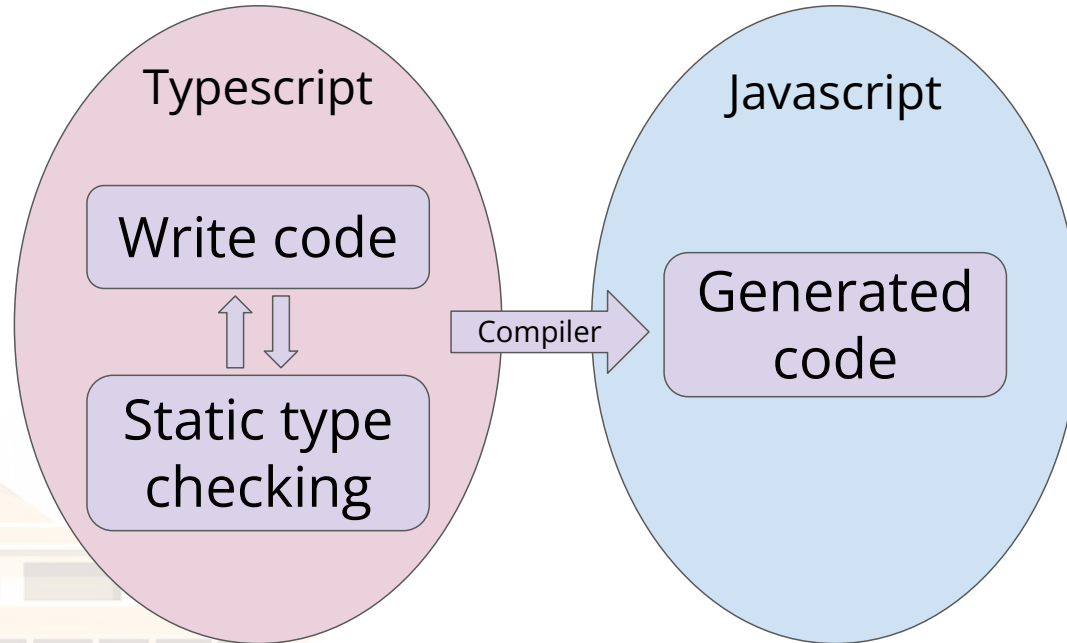
- TypeScript is a programming language that adds static type checking to JavaScript
- TypeScript is a superset of JavaScript, meaning that everything available in JavaScript is also available in TypeScript.
- the runtime behavior of TypeScript and JavaScript is identical



- **Interfaces**
- **Static Typing**
- **Generic Types**
- and more...



TS compiles to JS



Why do we care about static typing?

Here are some examples of issues that static typing can catch:

- Missing or unnecessary prop values
- Similarly named variables or functions
- Undefined & null value behavior - the "billion dollar mistake"
- Overloaded operators (e.g. addition, comparison)

```
type ProfileProps = {  
  | userId: string;  
};  
  
const Profile = (props: ProfileProps) => {  
  props.password;  
}
```



Why do we care about static typing?

Here are some examples of issues that static typing can catch:

- Missing or unnecessary prop values
- Similarly named variables or functions
- Undefined & null value behavior - the "billion dollar mistake"
- Overloaded operators (e.g. addition, comparison)

```
function callString(): string {  
  return "hi";  
}  
  
const calledString = "hello";  
  
let newString: string = callString;
```



Why do we care about static typing?

Here are some examples of issues that static typing can catch:

- Missing or unnecessary prop values
- Similarly named variables or functions
- **Undefined & null value behavior** - the "billion dollar mistake"
- Overloaded operators (e.g. addition, comparison)

```
function printString(s: string | null) {  
  console.log(s ?? "no string!");  
}
```



Why do we care about static typing?

Here are some examples of issues that static typing can catch:

- Missing or unnecessary prop values
- Similarly named variables or functions
- Undefined & null value behavior - the [“billion dollar mistake”](#)
- Overloaded operators (e.g. addition, comparison)

```
let sum: number = "5" + 4
```

Type 'string' is not assignable to type 'number'. ts(2322)

```
let sum: number
```

[View Problem \(⌘F8\)](#) No quick fixes available



Why do we care about static typing?

Here are some examples of issues that static typing can catch:

- Missing or unnecessary prop values
- Similarly named variables or functions
- Undefined & null value behavior - the ["billion dollar mistake"](#)
- Overloaded operators (e.g. addition, comparison)

```
let sum: number = "5" + 4
```

Type 'string' is not assignable to type 'number'. ts(2322)

```
let sum: number
```

[View Problem \(⌘F8\)](#) No quick fixes available



TypeScript with primitive types

- string, boolean, number

```
1 let message: string = "Hello world!";  
2 message = 1;
```

Type 'number' is not assignable to type 'string'. (2322)

[Peek Problem \(\F8\)](#) No quick fixes available



More Types

Arrays

```
let message: string[] = ["1", "2", "3"];
```

```
let message: Array<string> = ["1", "2", "3"];
```

```
const example: Array<string | number> = [1, 2, "three"];
```

Enums

```
type Color = "Red" | "Green" | "Blue";
```

```
let c : Color = ""
```

- Blue
- Green
- Red



Define your own type

- Define each property and its type
- Denote optional params with ?

```
type User = {  
  _id: string;  
  name: string;  
  is_admin?: boolean;  
};  
  
const user: User = {  
  _id: "555",  
  name: "Kenneth",  
};
```



Extend Types

Extending an interface

```
interface Animal {  
  name: string;  
}  
  
interface Bear extends Animal {  
  honey: boolean;  
}  
  
const bear = getBear();  
bear.name;  
bear.honey;
```

Extending a type via intersections

```
type Animal = {  
  name: string;  
}  
  
type Bear = Animal & {  
  honey: boolean;  
}  
  
const bear = getBear();  
bear.name;  
bear.honey;
```



Typed Functions

- You need to declare the type for function parameters and output!
- So users know exactly what type gets passed in and what type comes out.
- Function signature looks like (paramName: **paramType**): **returnType** => { }

```
1  const getComments = (id) => {  
2    |   return [];  
3  };
```

Javascript version:
a bit ~mysterious~

```
const getComments = (id: string): Comment[] => {  
  |   return [];  
};
```

TypeScript
version: a lot
clearer what's
going on!



Typed async functions

JavaScript version:
again, mysterious

```
const getComments = async (id) => {
```

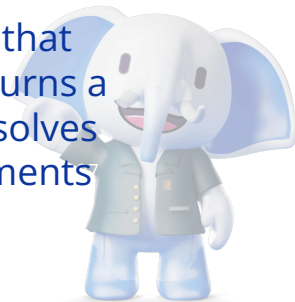
TypeScript
version!

```
const getComments = async (id: string): Promise<Comment[]> => {
```

Tells TypeScript this
is an async function

Tells TypeScript
that this function
takes in a string

Tells TypeScript that
this function returns a
Promise that resolves
to a list of Comments



Learn More About TypeScript

Official Web:

<https://www.typescriptlang.org/>

 **TypeScript**

Made with ♥ in Redmond,
Boston, SF & Dublin

 **Microsoft**

© 2012-2024 Microsoft



EOS

